

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessor - Microcontroller

(CO3009)

LAB 4: SCHEDULER

Tutors:	TS Lê Trọng Nhân Th.S Phan Văn Sỹ
Student:	Trương Thiên Ân
Student ID:	2310190
Class:	TN01



Table of contents

1	Exercises and Report	2
1.1	SCH_Init	2
1.2	SCH_Add_Task	2
1.2.1	delayInsert	3
1.3	SCH_Update	4
1.3.1	pushQueue	6
1.4	SCH_Dispatch_Tasks	6
1.4.1	popQueue	6
1.5	SCH_Delete_Task	6
2	Repository and Document Information	7

1 Exercises and Report

1.1 SCH_Init

```
1 void SCH_Init(void) {
2     head_waiting_list = NULL;
3
4     ready_head = 0;
5     ready_tail = 0;
6     ready_count = 0;
7     for (int i = 0; i < SCH_MAX_TASKS; i++) {
8         SCH_tasks_G[i].pTask = 0;
9         SCH_tasks_G[i].Period = 0;
10        SCH_tasks_G[i].RunMe = 0;
11        SCH_tasks_G[i].next = 0;
12        SCH_tasks_G[i].prev = 0;
13        SCH_tasks_G[i].TaskID = i;
14
15    }
16 }
```

Program 1: SCH_Init

Hàm Init này để:

1. Gán toàn bộ mảng các task về giá trị ban đầu (0)
2. Gán Queue chứa task sẵn sàng dispatch là rỗng
3. Khởi tạo danh sách delay (delay list)

1.2 SCH_Add_Task

```
1 void SCH_Add_Task(void (*pFunction)(), uint32_t DELAY, uint32_t PERIOD)
2 {
3     int index = 0;
4     while ((index < SCH_MAX_TASKS) && (SCH_tasks_G[index].pTask != NULL))
5     {
6         index++;
7     }
8     if (index == SCH_MAX_TASKS) return;
9
10    sTasks *new_task = &SCH_tasks_G[index];
11    new_task->pTask = pFunction;
12    new_task->Period = PERIOD;
13    new_task->TaskID = index;
14    new_task->RunMe = 0;
15
16    delayInsert(new_task, DELAY);
17 }
```

Program 2: SCH_Add_Task

Hàm Add này dùng để:

1. Gán tất cả các giá trị vào của task (hàm, delay, period, id, runMe)
2. Đặt biệt sẽ sử dụng hàm delayInsert để chèn vào danh sách delay

1.2.1 delayInsert

```
1 static void delayInsert(sTasks *new_task, uint32_t DELAY){
2     new_task->next = NULL;
3     new_task->prev = NULL;
4
5
6     if (head_waiting_list == NULL){
7         new_task->Delay = DELAY;
8         head_waiting_list = new_task;
9         return;
10    }
11
12
13    if (DELAY < head_waiting_list->Delay){
14        head_waiting_list->Delay -= DELAY;
15        new_task->next = head_waiting_list;
16        head_waiting_list->prev = new_task;
17        new_task->Delay = DELAY;
18        head_waiting_list = new_task;
19        return;
20    }
21
22
23    sTasks *current_task = head_waiting_list;
24    uint32_t temp_delay = DELAY;
25
26    while (current_task->next != NULL && temp_delay >= current_task->
Delay){
27        temp_delay -= current_task->Delay;
28        current_task = current_task->next;
29    }
30
31    if (temp_delay >= current_task->Delay && current_task->next == NULL
){
32
33        temp_delay -= current_task->Delay;
34        current_task->next = new_task;
35        new_task->prev = current_task;
36        new_task->Delay = temp_delay;
37    } else {
38
39        new_task->Delay = temp_delay;
40        current_task->Delay -= temp_delay;
41
42        new_task->next = current_task;
```

```
43     new_task->prev = current_task->prev;  
44     if (current_task->prev != NULL){  
45         current_task->prev->next = new_task;  
46     }  
47     current_task->prev = new_task;  
48 }  
49 }
```

Program 3: delayInsert

1. Hàm `delayInsert()` sử dụng **danh sách liên kết đôi dạng delay list** để tìm đúng vị trí chèn task mới theo giá trị `DELAY`.
2. Nếu danh sách đang rỗng thì task mới trở thành task đầu tiên với delay tuyệt đối ban đầu.
3. Nếu `DELAY` của task mới nhỏ hơn delay của task đầu:
 - Chèn task mới lên đầu danh sách.
 - Giảm delay của task đầu cũ đúng bằng `DELAY` của task mới.
4. Nếu `DELAY` lớn hơn delay node đầu:
 - Duyệt xuống danh sách, tại mỗi task sẽ trừ dần delay tương đối để tìm vị trí thích hợp.
5. Khi tìm được vị trí:
 - Nếu chèn vào cuối, delay của task mới chính là phần delay còn lại sau khi trừ.
 - Nếu chèn vào giữa:
 - Delay của task mới là delay còn lại sau khi duyệt.
 - Delay của task hiện tại (sau nó) bị trừ đi delay của task mới.
6. Nói gọn: chèn trước thì trừ delay của task phía sau; chèn giữa hoặc cuối thì giữ đúng tổng thời gian bằng cách phân bổ lại delay theo delay list.
 - Giả sử list hiện tại là:
 - A: Delay = 3
 - B: Delay = 3 $\Rightarrow$ A chạy ở tick 3, B chạy ở tick 6.
 - Thêm C với `DELAY = 4` (tức muốn C chạy ở tick 4):
 - Bước 1: `temp_delay = 4`, `current = A`. Vì $4 \geq 3$ nên trừ: `temp_delay = 4 - 3 = 1`, rồi nhảy sang `current = B`.
 - Bước 2: `current = B` là node cuối, và `temp_delay = 1 < B.Delay = 3` nên chèn C **trước** B.
 - Khi chèn:
 - * `C.Delay = 1`
 - * `B.Delay = 3 - 1 = 2`
 - Kết quả list dạng delta:
 - A(3), C(1), B(2)
 - Thời điểm tuyệt đối:
 - * A: 3
 - * C: $3 + 1 = 4$
 - * B: $3 + 1 + 2 = 6$

1.3 SCH_Update

```
1 void SCH_Update(void){
```

```
2   if (head_waiting_list == NULL) return;
3
4   if (head_waiting_list->Delay > 0) {
5       head_waiting_list->Delay--;
6   }
7
8   while (head_waiting_list != NULL && head_waiting_list->Delay == 0)
9   {
10       sTasks *task = head_waiting_list;
11       head_waiting_list = head_waiting_list->next;
12       if (head_waiting_list != NULL) {
13           head_waiting_list->prev = NULL;
14       }
15
16       task->next = NULL;
17       task->prev = NULL;
18
19       task->RunMe++;
20       pushQueue(task->TaskID);
21   }
22
23   // sTasks *task = head_waiting_list;
24   //
25   // head_waiting_list = head_waiting_list->next;
26   // if (head_waiting_list != NULL) {
27   //     head_waiting_list->prev = NULL;
28   // }
29   //
30   // task->next = NULL;
31   // task->prev = NULL;
32   //
33   // task->RunMe++;
34   // pushQueue(task->TaskID);
35 }
```

Program 4: SCH_Update

Hàm này chịu trách nhiệm việc trừ Delay và nếu Delay bằng 0 thì + RunMe đồng thời push id của task sẵn sàng chạy vào ready Queue

Ở hàm này em có 2 trường hợp nhờ Thầy xem xét ạ:

- 1: Sử dụng while để duyệt qua xem có các sự kiện nào đồng thời diễn ra hay không -> nếu có thì sẽ push 1 lần vào Ready Queue. Ở trường hợp này em thấy mô phỏng sẽ đi nhanh hơn vì các task thực hiện cùng 1 lúc đã được push và Queue trong cùng 1 tick nên khi dispatch trong while(1) gọi ra thì gọi ra được hết.
- 2: Chỉ xét tới thẳng đầu, nếu delay ==0 thì cập nhật RunMe và push vào trong Queue. Ưu điểm O(1) theo yêu cầu đề bài nhưng vì mỗi tick chỉ có thể push được 1 task có RunMe == 1 nếu những task khác cần phải thực hiện cùng thời gian với task này thì phải đợi tick sau (10ms) nên mô phỏng sẽ thấy sự chậm rõ hơn so với trường hợp trên

1.3.1 pushQueue

```
1 static void pushQueue(uint32_t id){
2     if (id > SCH_MAX_TASKS){
3         return;
4     }
5     ready_q[ready_tail] = id;
6     ready_tail = (ready_tail + 1) % SCH_MAX_TASKS;
7     ready_count++;
8 }
```

Program 5: pushQueue

Hàm này chịu trách nhiệm để push id của task vào Queue

1.4 SCH_Dispatch_Tasks

```
1 void SCH_Dispatch_Tasks(void){
2     uint32_t id;
3     if (popQueue(&id)){
4         SCH_tasks_G[id].RunMe--;
5         (*SCH_tasks_G[id].pTask)();
6         delayInsert(&SCH_tasks_G[id], SCH_tasks_G[id].Period);
7     }
8 }
```

Program 6: SCH_Dispatch_Tasks

Hàm này được gọi trong While(1) để gọi các task ra thực hiện khi cờ RunMe == 1. Hàm này xét coi popQueue(&id) có task nào sẵn sàng được push vô chưa, kiểm tra RunMe nếu RunMe > 0 thì trừ RunMe, gọi hàm thực hiện task và đưa task đó vào lại danh sách delay.

1.4.1 popQueue

```
1 static int popQueue(uint32_t *id){
2     if (ready_count == 0){
3         return 0;
4     }
5     *id = ready_q[ready_head];
6     ready_head = (ready_head + 1) % SCH_MAX_TASKS;
7     ready_count--;
8     return 1;
9 }
```

Program 7: popQueue

Hàm này để lấy một id của 1 task ra khỏi Queue để chuẩn bị hiện thực.

1.5 SCH_Delete_Task

```
1 void SCH_Delete_Task(uint32_t TaskID){
2     sTasks *deleted = &SCH_tasks_G[TaskID];
3     if (deleted->pTask == NULL){
4         return;
5     }else{
6         if (deleted->next != NULL) {
7             deleted->next->Delay += deleted->Delay;
8         }
9     }
10
11     if (deleted->prev != NULL){
12         deleted->prev->next = deleted->next;
13     }else{
14         if (head_waiting_list == deleted){
15             head_waiting_list = deleted->next;
16         }
17     }
18     if (deleted->next != NULL){
19         deleted->next->prev = deleted->prev;
20     }
21
22     deleted->pTask = NULL;
23     deleted->next = NULL;
24     deleted->prev = NULL;
25     deleted->RunMe = 0;
26 }
```

Program 8: SCH_Delete_Task

Hàm này có chức năng chính là xóa đi 1 task, và nếu task bị xóa có nằm trước hay giữa 2 task nào thì tiến hành cộng delay cho task sau

2 Repository and Document Information

Root Repository (All labs: [HCMUT_STM32_2310190_LAB](#)

Lab 4 Source code: [2310190_LAB4](#)

Link video demo [2310190_LAB4_VIDEO_DEMO](#)

Author: Trương Thiên Ân

Last update: November 20th, 2025